

AI Security 1.4. Prompt Engineering

Task 1 - Introduction

This room will cover some LLM fundamentals before showing you how to effectively use an LLM, whether that's for security-related tasks or testing a model's behaviour against malicious prompts.

Learning Objectives

- Understand what tokens are and how LLMs process text
- Grasp nondeterminism and why LLMs produce variable outputs
- Control model behaviour using temperature, max tokens, and top-p
- Understand the four pillars of effective prompt engineering
- Recognise the difference between system and user prompts
- Understand key prompting techniques

Task 2 - LLM Fundamentals

Token: the term for the smallest units that an LLM breaks text into in order to process it. LLM models don't read text the way you do. When you type "Hello, how are you?" the model breaks it into **tokens**, the smallest units it can understand. Think of tokens as the "words" that AI actually sees, though they don't match our definition of words. A token is roughly equivalent to 3-4 characters, meaning most English words are 1-2 tokens. The model converts each token into a unique number (an ID). Your message "Hello, how are you?" might become the numbers [15496, 11, 703, 527, 499]. The model only works with these numbers, using them to predict what number (token) should come next.

nondeterminism: outputs vary even with identical inputs. Unlike traditional software, where the same input always produces the same output (deterministic behaviour), LLMs introduce randomness when predicting the next token.

When you send a prompt to an LLM, you're not just typing words into a black box; you're triggering a **probability machine** that weighs hundreds of thousands of potential next words.

Temperature is the most important parameter you'll touch. This is a numerical value, commonly ranging from 0.0 - 2.0 (which can differ between providers) that controls how "adventurous" the model is when picking its next word.

What parameter would you set to 0.0 to make an LLM behave as close to deterministic as possible?

Temperature

Temperature Range	Behaviour	Use Case
0.0 – 0.3	Always picks the most probable token; closest to determinism	Code generation, data extraction, factual Q&A
0.7 – 1.0	Samples from a wider distribution; more variety and creativity	Brainstorming, storytelling, marketing copy
1.2 – 1.5	Coherence begins to break down; unpredictable outputs	Experimental use only
1.5+	Low-probability tokens dominate; outputs can feel "drunk"	Avoid for most tasks

Max Tokens: The Length Limiter

Max tokens caps how **long** the response can be. One token roughly equals 0.75 English words, so 100 tokens usually equals about 75 words. Set this too low and the model cuts off mid-sentence. Set it too high and you pay for rambling you don't need. Consumer models, like those on paid plans like OpenAI, charge per token, so **controlling length is a cost-control measure**.

Top-P: The Alternative Randomness Dial

Top-p (nucleus sampling) is temperature's cousin. Think of it like this: instead of turning up or down the randomness across all possible words, top-p **sets a shortlist**. Set top-p to 0.9, and the model only considers words that together account for 90% of the likely options. The weird, unlikely tail gets cut off entirely.

The higher the value, the bigger the shortlist; the lower the value, the more restricted the model's choices.

In general, it is advised to adjust **temperature OR top-p**, but not both. Using both simultaneously creates unpredictable interactions because you're stacking two randomness controls. Temperature is good for most tasks; it's simpler, more intuitive, and widely understood. Top-p is better when you need the model to adapt its creativity based on context.

Top-p restricts which tokens the model considers by limiting selection to a cumulative probability mass.

Context Window: The Memory Limit

Every model has a **context window**: its maximum "working memory" measured in tokens. Modern LLMs range from 8k tokens (older GPT-3.5) to 1M+ tokens (Gemini 1.5 Pro). Claude 3.5 Sonnet handles 200k tokens, enough for 150,000 words or 500 pages of text. Exceed this limit and the model silently truncates earlier context. It literally forgets the start of your conversation.

context window term describes the maximum working memory of an LLM, measured in tokens.

Try this experiment: prompt `"Explain SQL injection"` with temperature 0.0, then 1.0. At 0.0, you'll get a textbook definition. At higher temperatures, expect creative tangents, maybe even fictional anecdotes. Adjust max tokens from 50 to 500; watch the explanation expand from a tweet to an essay. These parameters don't change what the model knows; they change how it expresses that knowledge. Master them and you become a graduate prompt engineer.

Answer the questions below

What is the term for the smallest units that an LLM breaks text into in order to process it?

Tokens

What parameter would you set to 0.0 to make an LLM behave as close to deterministic as possible?

Temperature

What parameter restricts which tokens the model considers by limiting selection to a cumulative probability mass?

Top-p

What term describes the maximum working memory of an LLM, measured in tokens?

Context window

Task 3 - The Anatomy of a Prompt

Experts often break prompts into clear components (or pillars): the core **instruction** (task to perform), relevant **context** (background information), the desired **output format** (structure/style), and any **constraints** (rules or limits). When all these elements work together, the model has a well-defined framework for generating accurate, on-target responses.

Instruction (task): This is the core command or action you want from the AI, expressed with a clear verb.

Context (background): Context provides the AI with relevant information or a scenario so it understands the situation and perspective. This can include domain details, objectives, or background documents.

Output format (structure): Specify how you want the answer to look. This could mean asking for bullet points, a numbered list, a table, code blocks, a JSON object, or a certain word count.

Constraints (boundaries): These are any rules or limits you impose on the response. Constraints guide the model to follow specific boundaries; for example, forbidding certain topics, enforcing a style guide, or mandating a tone. They ensure output aligns with your needs. For example, "Write an academic report on IoT devices, provide citations in MLA format, and include a bullet-pointed summary section at the end (do NOT exceed 5 bullets)". It specifies rules or limits imposed on the model's response, such as enforcing a tone or forbidding certain topics.

Answer the questions below

Which pillar instructs the model on how the answer should be structured, such as bullet points or a JSON object?

Output Format

Which pillar specifies rules or limits imposed on the model's response, such as enforcing a tone or forbidding certain topics?

Constraints

Which pillar provides the AI with relevant background information or scenario so it understands the situation?

Context

Which pillar of prompt engineering defines the core command or action you want the AI to perform?

Instruction

Task 4 - System vs User Prompts

In LLM applications, **system prompts** (a.k.a. system messages) are developer-defined, persistent instructions that set the assistant's role, tone, and hard rules. They define the model's behaviour, role, and constraints at the application level and remain constant across sessions.

	System Prompt	User Prompt
Set by	Developer / application	End user
Nature	Immutable, constant	Dynamic, session-specific
Purpose	Establishes identity, rules, and safety boundaries	Carries task-specific requests and data
Example	"Never execute code. Always be helpful and professional."	"Summarise this document for me."
Priority	High-priority context that shapes overall behaviour	Acted on within the system prompt's constraints

This **instruction hierarchy** sounds solid in theory: system prompts set the rules, user prompts provide the questions, and the model follows the system's constraints while answering the user. However, there's a fundamental challenge that makes this separation difficult to enforce in practice. LLMs process everything as text. Regardless of whether something is labelled "system", "developer", or "user", the model ultimately sees a single sequence of tokens.

Understanding this limitation is crucial because it reveals the attack surface covered in the upcoming Prompt Security module. The foundation for this attack surface is that when system and user inputs merge into a single text stream, clever adversaries can craft user input that mimics or conflicts with system instructions. The model, trained to be helpful and follow instructions, may struggle to determine which directives take precedence, especially when user input is phrased persuasively or formatted to look authoritative.

Answer the questions below

What type of prompt is developer-defined, persistent, and remains constant across all sessions?

System prompt

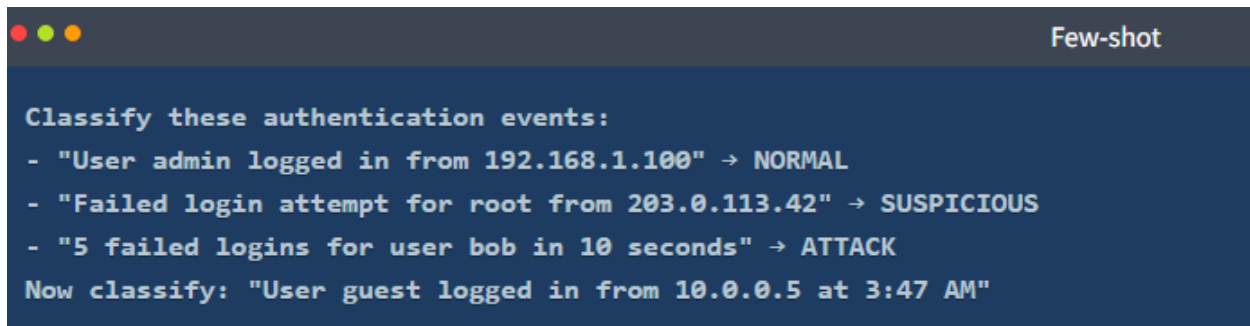
What is the term for the intended order of priority between system and user instructions in an LLM application?

Instruction hierarchy

Task 5 - Advanced Prompting Techniques

The Shot Spectrum

The term "shot" refers to training examples you provide within your prompt. **Zero-shot** prompting gives the model a task with no examples, relying entirely on its pre-trained knowledge. **One-shot** provides a single example to clarify expectations. **Few-shot** includes 2-5 examples so the model recognises patterns. This is called **in-context learning**: the model learns directly from examples embedded in your prompt rather than through traditional training.

A terminal window with a dark blue background and white text. The title bar at the top has three colored dots (red, yellow, green) on the left and the text 'Few-shot' on the right. The text inside the terminal reads: 'Classify these authentication events:' followed by three bullet points: '- "User admin logged in from 192.168.1.100" -> NORMAL', '- "Failed login attempt for root from 203.0.113.42" -> SUSPICIOUS', and '- "5 failed logins for user bob in 10 seconds" -> ATTACK'. The final line reads: 'Now classify: "User guest logged in from 10.0.0.5 at 3:47 AM"'.

```
Classify these authentication events:  
- "User admin logged in from 192.168.1.100" -> NORMAL  
- "Failed login attempt for root from 203.0.113.42" -> SUSPICIOUS  
- "5 failed logins for user bob in 10 seconds" -> ATTACK  
Now classify: "User guest logged in from 10.0.0.5 at 3:47 AM"
```

Chain-of-Thought (CoT) prompting gets reasoning. Introduced by Google researchers in 2022, CoT asks models to break down complex tasks into intermediate steps, mimicking how humans solve multi-step problems. Instead of jumping to conclusions, the model "thinks out loud".

CoT significantly improves performance on arithmetic, logic, puzzles, and multi-hop reasoning when used as a prompt technique. We now have reasoning models (like OpenAI's o1 series, Anthropic's thinking mode, etc.) which essentially have this built into them. There is a catch, though: CoT prompting only works well with models above 100B parameters. Smaller models have been known to generate reasoning chains that look coherent but lead to wrong answers.

Prompt Templates

This is more of a technique for streamlining AI tool usage. Templates are standardised prompt structures for recurring tasks. Instead of manually typing prompts or using the tools to generate prompts for tasks you need help with regularly, especially if a prompt has been iterated on and engineered to reliably generate excellent output, it makes sense to save and catalogue the template for future use.



```
Review this [LANGUAGE] code for [VULNERABILITY_TYPES]:
Context: [PURPOSE]
Code: [CODE_BLOCK]
Output format:
1. Vulnerabilities found (severity: critical/high/medium/low)
2. Affected lines
3. Remediation steps
4. Example secure code
```

- **Zero-shot:** Simple, well-defined tasks where instructions are clear
- **One-shot:** Format clarification or style guidance needed
- **Few-shot:** Complex patterns, domain-specific outputs, multiple edge cases
- **Chain-of-Thought:** Multi-step reasoning, security analysis requiring justification, debugging complex logic
- **Templates:** Repeatable tasks, team standardisation, quality control

Answer the questions below

What is the term for the prompting technique introduced by Google researchers in 2022 that asks models to break tasks into intermediate reasoning steps?

Chain-of-Thought

What prompting technique involves providing no examples and relying entirely on the model's pre-trained knowledge?

Zero-shot

What prompting technique involves saving and reusing a standardised prompt structure for recurring tasks?

Prompt templates

What simple phrase can be added to a prompt to trigger Zero-shot Chain-of-Thought reasoning?

Let's think step by step

Task 6 - Challenge

You've covered the theory, now it's time to put it into practice. In this challenge, you'll write prompts for real security tasks and be graded on your technique.

- **PromptSec** will give you a challenge, specifying a technique and a security task to write a prompt for
- Your prompt gets **graded out of 10** based on how well you apply the technique. You get the corresponding amount of points
- Rack up **40 points** to earn the flag
- Stuck? Ask PromptSec for a hint. It won't write the prompt for you, but it'll point you in the right direction

| What's the flag?

THM{Pr0mpt_3ng1neer}

Task 7 - Conclusion

At the start of the room, we set out to equip you with the foundational skills needed to use LLMs effectively before diving into their security implications. You've learned how to structure prompts, understand the instruction hierarchy, and control model behaviour through parameters. Here's a recap of what has been covered:

- Effective prompts follow four pillars: **clear instructions**, **relevant context**, **specified output format**, and **defined constraints**, transforming vague requests into precise directives.
- **System prompts** set persistent rules and behaviour, while **user prompts** provide task-specific queries. However, LLMs process both as a single text stream, creating probabilistic security boundaries rather than guaranteed ones.
- **Tokens** are the fundamental units LLMs process (roughly 3-4 characters each), with models converting text into numerical token IDs that they use to

predict what comes next.

- LLMs are **nondeterministic** by nature, meaning identical inputs can produce different outputs due to both intentional randomness controls (like temperature) and underlying computational variations.
- Parameters like **temperature**, **max tokens**, and **top-p** allow you to control model behaviour, balancing between precision and creative variety depending on your task.